

[illegible]

[illegible]

```

109         this.host            = "localhost";
110         this.port             = 5432;
111         this.databaseName     = "app_db";
112         this.username         = "admin";
113         this.password         = "s3cr3t";
114         this.maxPoolSize      = 10;
115         System.out.println(" [DatabaseConfig] Instance created (private con...
116     }
117
118     // & Public static accessor - lazy init
119     public static synchronized DatabaseConfig getInstance() {
120         if (instance == null) {
121             instance = new DatabaseConfig();    // created only the FIRST time
122         }
123         return instance;
124     }
125
126     public String getHost()          { return host; }
127     public int   getPort()           { return port; }
128     public String getDatabaseName() { return databaseName; }
129     public String getUsername()      { return username; }
130     public int   getMaxPoolSize()    { return maxPoolSize; }
131
132     @Override
133     public String toString() {
134         return String.format("DatabaseConfig{host='%s', port=%d, db='%s', us...
135                             host, port, databaseName, username, maxPoolSize);
136     }
137 }
138
139 // %%%%%%%%%%%%%%%
140 // 4 . ConnectionPool %% SINGLETON %%
141 // Manages a pool of Connection objects.
142 // Uses DatabaseConfig singleton for configuration.
143 // %%%%%%%%%%%%%%%
144 class ConnectionPool {
145
146     // & Static field that holds the single
147     private static ConnectionPool instance;
148
149     private int activeConnections;
150     private final List<Connection> connectionList;
151     private final DatabaseConfig config;        // uses the other Singl...
152
153     // & Private constructor - prevents dir
154     private ConnectionPool() {
155         this.config = DatabaseConfig.getInstance(); // reuses th...
156         this.connectionList = new ArrayList<>();
157         this.activeConnections = 0;
158         System.out.println(" [ConnectionPool] Instance created (maxPoolSize...
159     }
160
161     // & Public static accessor - lazy init
162     public static synchronized ConnectionPool getInstance() {

```

[illegible]

```

217 public void run() {
218     System.out.println("\n=====");
219     System.out.println("  " + appName + " v" + version + " starting...");
220     System.out.println("=====\\n");
221
222     // %% Retrieve Singletons %%%%%%%%%%
223     System.out.println("%o Fetching Data
DatabaseConfig dbConfig = DatabaseConfig.getInstance();
224
225     System.out.println(" Config: " + dbConfig);
226
227     System.out.println(" \\n%o Fetching Da
DatabaseConfig dbConfig2 = DatabaseConfig.getInstance();
228
229     System.out.println(" Same instance: " + (dbConfig == dbConfig2)); ...
230
231     System.out.println(" \\n%o Fetching Co
ConnectionPool pool = ConnectionPool.getInstance();
232
233
234     System.out.println(" \\n%o Fetching Co
ConnectionPool pool2 = ConnectionPool.getInstance();
235
236     System.out.println(" Same instance: " + (pool == pool2)); ...
237
238     // %% Use pool to obtain a connecti
239     System.out.println(" \\n%o Borrowing a
Connection conn1 = pool.getConnection();
240
241
242     System.out.println(" \\n%o Executing a
if (conn1 != null) {
243         ResultSet rs = conn1.executeQuery("SELECT id, name, role FROM us...
244         System.out.println(" Query returned " + rs.getRowCount() + " ...
245         while (rs.next()) {
246             System.out.printf(" id=%-3s name=%-8s role=%s\\n",
247                 rs.getString("id"), rs.getString("name"), rs.getStri...
248         }
249         rs.close();
250     }
251 }
252
253     System.out.println(" \\n%o Borrowing a
Connection conn2 = pool.getConnection();
254
255
256     System.out.println(" \\n%o Pool active
257
258     // %% Release connections and shutd
259     System.out.println(" \\n%o Releasing c
pool.releaseConnection(conn1);
260
261
262     System.out.println(" \\n%o Shutting do
pool.shutdown();
263
264
265     System.out.println("\n=====");
266     System.out.println(" Application finished.");
267     System.out.println("=====");
268 }
269
270 // %%% main %%%%%%%%%%

```

```
271     public static void main(String[] args) {
272         Application app = new Application("SingletonDemo", "1.0.0");
273         app.run();
274     }
275 }
276
```